Trabajo Practico N° 5 – Programación 4

"Gestión de Estado con useReducer + Context"

"¿Qué pasa cuando la lógica del estado empieza a crecer mucho?"

- La lógica se vuelve dispersa
- Se hace difícil escalar la aplicación
- Se hace difícil el mantenimiento

useReducer

useReducer es un hook que permite manejar el estado mediante acciones. Centraliza TODA la lógica en un solo lugar. En lugar de modificar el estado directamente, enviamos órdenes (acciones)

Objetivos del Práctico:

- Reemplazar useState por useReducer
- Centralizar la lógica de participantes
- Integrar useReducer con Context
- Mantener persistencia con JSON y Database

Concepto clave

"En lugar de modificar el estado directamente, enviamos acciones que el reducer procesa."

En lugar de modificar el estado directamente:

setEstado(nuevoEstado)

usamos:

dispatch({ type: "ACCION", payload: dato })

Componentes del hook (patrón)

reducer

Es una función que decide cómo cambia el estado.

- Recibe el estado actual
- Recibe una acción
- Devuelve un nuevo estado

action

Es un objeto que describe qué queremos hacer. Ejemplo:

```
{
  type: "AGREGAR",
  payload: participante
}
```

dispatch

Es la función que envía la acción al reducer.

```
dispatch({ type: "AGREGAR", payload: nuevo });
```

Flujo completo

```
Componente
    ↓
dispatch(action)
    ↓
reducer(state, action)
    ↓
nuevo estado
    ↓
React renderiza
```

Estructura

```
src/
 ├── context/
 │    └── ParticipantesContext.tsx
 ├── reducers/
 │    └── participantesReducer.ts
 ├── models/
 │    └── Participante.ts
 ├── components/
 │    ├── Formulario.tsx
 │    └── ParticipanteCard.tsx
 └── Home.tsx
```

En participantesReducer deberá implementar la lógica necesaria que permita ejecutar las acciones asociadas a la aplicación, principalmente aquellas relacionadas a la manipulación de los datos y peticiones cliente/servidor

Por ejemplo las siguientes acciones:

```
export type Action =

 | { type: "GET_PARTICIPANTES"; payload: Participante[] }
 | { type: "AGREGAR"; payload: Participante }
 | { type: "ELIMINAR"; payload: number }
 | { type: "RESET"; payload: Participante[] }
 | { type: "EDITAR"; payload: Participante }
 | { type: "SET"; payload: Participante[] };
```

Deberá agregar una nueva funcionalidad que permita editar un participante, al hacer click sobre el botón "Editar" los datos del participante se deberán cargar automáticamente en el Formulario, y tras la modificación de alguno de sus datos y su correspondiente almacenamiento se actualizara la tarjeta con la nueva información. Modifique el backend para soportar esta nueva funcionalidad.